Programowanie gier komputerowych

Organizacja zajęć

inż. Mateusz Połeć

Semestr zimowy 2026/2027

Spis treści

1. Warunki zaliczenia
2. Kontakt
3. Cel zajęć
4. Jak będziemy pracować?
5. Nad czym będziemy pracować?
6. Literatura
7. Twórcy internetowi

Warunki zaliczenia

Zaliczenie odbywa się na podstawie obecności, list zadań i projektu gry.

- Listy zadań: Opcjonalne (choć wskazane!), podnoszą ocenę w górę.
- Demonstracja prototypu gry (może być z poprzednich przedmiotów/lat), podnosi ocenę w górę.
- Obecność: do trzech (3) nieobecności bez podania przyczyny. Obecności są warunkiem koniecznym, ocena w górę za listy zadań i prototyp gry nie ratują osoby z większą ilością nieobecności.

Kontakt

- Mail: mateusz.polec@uwr.edu.pl
- Teams: Mateusz Połeć / mateusz.polec@uwr.edu.pl.
- Prywatny mail: mpolec2000@gmail.com
- Discord: kolegamateusz_

Cel zajęć

Głównym celem tego kursu jest wprowadzenie do fundamentalnych zagadnień z zakresu programowania gier. W tym celu prezentowane będą treści wprowadzające do tematu zajęć, analiza kodu innych gier oraz praca własna, opcjonalna, w postaci list zadań.

Plan na zajęcia – jak będziemy pracować?

1. Wprowadzania do zagadnień tematu zajęć.
2. Analiza kodu innych gier.
3. Przykładowe rozwiązania list zadań z omówieniem.
4. Technologicznie pracujemy w C++ z wykorzystaniem biblioteki SFML. System budowania dowolny (CMake/Premake + Visual Studio).

Plan na zajęcia – nad czym będziemy pracować? Pt. 1

1. Wprowadzenie do programowania gier, dostępne silniki gier, konfiguracja projektu (SFML+Premake+Visual Studio i inne najpotrzebniejsze biblioteki).
2. System wydarzeń (eventy), Systemy wspomagające: Logger, Assercje, Crash Dump.
3. Filesystem i obsługa plików.
4. Obsługa I/O.
5. ECS (Entity-Component System).
6. Systemy gracza i kamera.
7. Animacje (Porównanie różnych technik, implementacja dwuwymiarowych sprite-based animacji).
8. World streaming, poziomy.

Plan na zajęcia – nad czym będziemy pracować? Pt. 2

1. Matematyka w grach (system kartezjański, wektory, macierze)
2. Systemy fizyki (raytrace, kolizje, mass, rigid body).
3. Sztuczna inteligencja (Path-finding, navmesh)
4. Sztuczna inteligencja (Black-boards, Behaviour Trees).
5. Interfejsy użytkownika (UI)
6. GPU, Shadery, Światła.
7. Implementacja prostej gry na bazie fundamentu, który zbudowaliśmy.

Literatura

- Effective Modern C++ – Scott Meyers
- Game Engine Architecture – Jason Gregory
- Game Programming Patterns – Robert Nystorm
- Real-Time Rendering (4th Edition) – Tomas Akenine-Moller
- Game Programming Gems (seria) – Andrew Kirmse
- Game Engine Black Book: DOOM / Wolfenstein 3D – Fabien Sanglard
- 3D Math Primer for Graphics and Game Development – Fletcher Dunn i Ian Parbery
- Unreal Engine 5 Game Development Cookbook – John P. Doran

Twórcy internetowi

- The Cherno - YouTube (C++/Custom game engines)
- Jason Turner – YouTube (C++)
- tokyospliff – YouTube (C++/Custom game engines)
- Travis Vroman – YouTube (C/Custom game engines)
- Tom Looman – YouTube (C++/Unreal Engine)
- Ryan Laley – YouTube (Unreal Engine)
- Matt Aspland – YouTube (Unreal Engine)